Kuku AI Generalist Assignment · Problem 1

Promo Finder

An AI tool that reads microdrama screenplays and finds the best 20–60 second promo clips, ranked by what to post first.

50
episodes in the input pack

3,727
timestamped lines parsed, 0 warnings

~103 min
of footage covered

52
AI requests for a full show (about 180K input tokens)

8
AI providers, switchable with one setting

15
offline tests, including a show in a new format

Contents

1 High-level overview
2 What we built
3 How it works, step by step
4 System design
5 Key design decisions and trade-offs
6 Quality checking and known failures
7 Scale, cost and reliability
8 Tech stack and project layout
9 How to run it
10 Interview questions with model answers

Built in Python (FastAPI) with a React UI. Default model: Google Gemini Flash via the Interactions API.
The input pack is Kuku's confidential content. This guide quotes it only in a few short examples.

1 High-level overview

The problem

Kuku makes AI-generated **microdramas**: short vertical series with 1–2 minute episodes. New viewers come in through **promos**, 20–60 second clips cut from the show and posted on Instagram, TikTok and YouTube. Today an editor watches every episode and picks promo moments by hand, one show at a time, and new shows come out every week. That doesn't scale.

What a good promo needs (from the brief):

1. **A hook** in the first 3 seconds that stops a scrolling viewer: a slap, a reveal, a shocking line.
2. **A cliffhanger ending**: it cuts at an unresolved moment, so the viewer has to find out what happens.
3. **It stands alone**: it makes sense to someone who has never seen the show.

Our solution in one paragraph

Promo Finder reads a show's timestamped screenplays and returns a **ranked shortlist of promo clips**. For each one it gives the episode and exact start/end of every segment, the hook, the cliffhanger, one line on why it works, and where it should rank in the posting order. Cheap **rules** shortlist strong moments. An **LLM** reads each episode and picks and explains the best clips. **Code** then checks every timestamp and duration, so the AI can never invent footage. An **editor** approves or rejects each pick in a simple web UI and exports the approved ones as edit lists for Premiere or DaVinci Resolve.

Who uses it, and how

1 · Upload
The editor drops the screenplay zip. The tool reports how many episodes it found and what a run will cost in AI requests and minutes, before anything is spent.

2 · AI finds moments
It picks hook-to-cliffhanger clips per episode, builds multi-episode trailers, checks every timestamp, and ranks the results.

3 · Review & export
The editor reads each pick's script excerpt, approves or rejects it with a reason, and exports an EDL and ffmpeg script.

Scope: what it does and doesn't do

In scope	Out of scope (as the brief says)
Finding promo moments and exact start/end points	Video editing, music and text overlays
Single clips, multi-cut clips and cross-episode trailers	Generating new footage
Hook, cliffhanger and reason for each pick	Reading the video itself (it only reads the screenplay)
Ranking and posting order	Publishing to social platforms
Quality checks and honest failure reporting	

Assumptions we made where the brief was silent

- **Script time = video time.** The screenplay timestamps match the final cut.
- **Promos are acquisition ads for strangers.** "Makes sense alone" carries a lot of weight, and clips from early episodes (1–10) get a small bonus, because a new viewer starts at episode 1.
- **The hook must be a line, action or sound.** A camera move alone doesn't count.
- **Never cut mid-line.** Clip edges always snap to real line boundaries.
- **Spoilers are fine.** In this genre the twist (for example "he is secretly the Duke") is the selling point.
- **Montages may only use code-extracted "beats"**, so the AI can't make up a timestamp.

2 What we built

Five deliverables that work together. All of them run locally, and the AI is optional.

🧠 **Promo engine** (Python package `promo_finder`)

Parser → rule signals → LLM passes → validation → ranking → reports. It handles any number of episodes and falls back to rules if the AI fails.

⌨️ **Command-line tool** (`cli.py`)

Batch runs, a free `--dry-run` cost check, a `--max-calls` budget, an episode subset, and switching off montages or the judge.

🖥️ **Web app** (FastAPI + React)

Explains the product, lets you drag in a zip, pick a model from a dropdown, watch live progress with a timer, review promo cards, approve or reject with reasons, and export.

🎬 **Editor export** (`export.py`)

A CMX3600 **EDL** per promo (imports into Premiere Pro and DaVinci Resolve) and an **ffmpeg** script that cuts and joins the clips. Tested: a 29.5s + 9.5s promo came out at exactly 39.0s.

📊 **Quality + feedback** (`quality_check.py`, `feedback.py`)

Hard-rule compliance, coverage, flagged picks, and precision from editor decisions. It also shows which score separates good picks from bad, as evidence for tuning.

✅ **Tests + demo kit**

15 offline unit and integration tests, an original test show in a different script format, and a 5-minute interview demo script.

What one promo looks like (output)


```
{
  "rank": 1, "id": "P012", "kind": "single",
  "title": "Jilted at the altar",
  "segments": [ { "episode": 1, "start": "00:39.10", "end": "01:14.30", "duration": 35.2 } ],
  "hook": "\"Because he is marrying me.\" (her sister, holding the groom's hand)",
  "cliffhanger": "Juliana asks \"How long?\" and the scene cuts before anyone answers",
  "why": "Instant betrayal a stranger understands in 3 seconds; cuts on an open question",
  "scores": { "hook": 9, "cliffhanger": 8, "standalone": 9, "emotion": 9 },
  "final_score": 0.84, "rank_reason": "Strongest premise; post first to set up the show",
  "flags": [], "excerpt": ["E01 00:39.10 LADY IN GOLD GOWN (Gloating, Smug): Because he is
marrying me.", "..."]
}
```


An illustrative example of the format. Real values come from `outputs/<show>/promos.json`.

Output files for every run

File	Purpose
`promos.md`	Human-readable ranking with script excerpts, flags, rejected picks and a run log
`promos.csv` / `promos.json`	Editor spreadsheet / everything, machine-readable
`raw_llm/*.json`	Every prompt and its **unedited** model response (the brief's "raw outputs"). Also used as a cache

`approvals.json`	Editor decisions (approved or rejected, plus a reason). This becomes labelled data
`quality.md`	The quality report

3 How it works, step by step

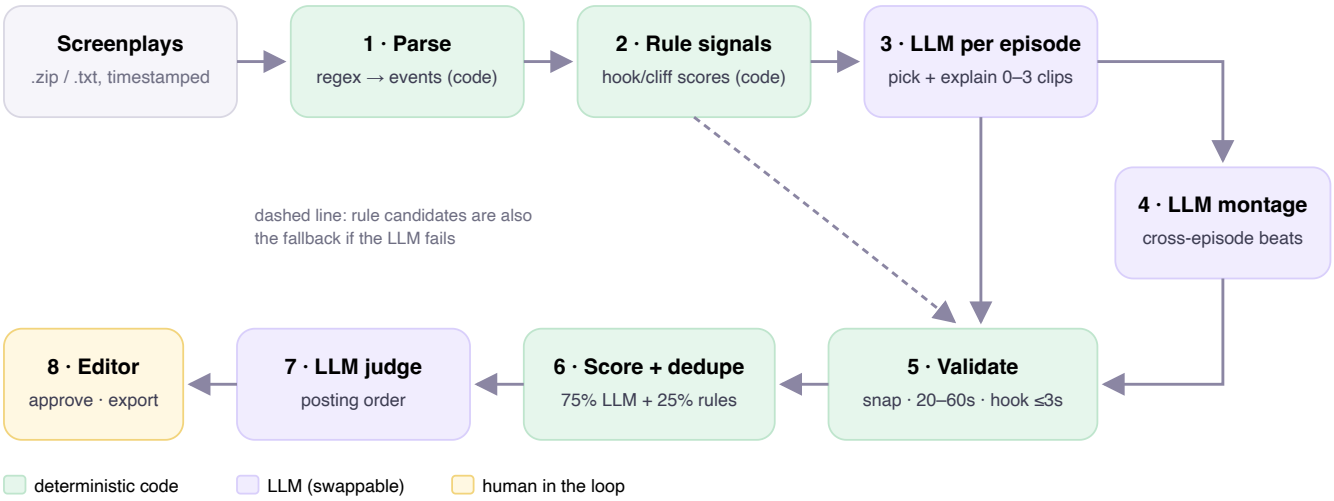


Figure 1. The promo pipeline. Green steps are code, purple steps are AI, and the yellow step is the person who signs off.

Step	What happens	Why this way
1 · Parse parser.py	Regex reads episode number, logline, scenes and every block (CAMERA / ACTION / SOUND FX / CHARACTER (emotion)) with start/end seconds. Accepts MM:SS or HH:MM:SS, bold or plain labels, and several timestamp styles. Bad blocks are logged and skipped.	The input is structured, so code is exact, free and fast. Being forgiving means a new show won't crash it.
2 · Rule signals signals.py	Each line gets a **hook score** (violence and reveal words, intense emotion tags, short punchy lines) and a **cliffhanger score** (stinger sound cues, "cut to black", questions and threats, last 12s of the episode). It generates 20–60s windows from strong hooks to strong endings and keeps the top 5 per episode.	Gives the LLM a shortlist, acts as a sanity score, and lets the tool work with no AI at all.
3 · LLM per episode	One request per episode: the full timestamped script, the logline, and the rule candidates. Returns JSON with 0–3 promos: segments, hook, cliffhanger, why, 1–10 scores (hook, cliffhanger, standalone, emotion), context a stranger would lack, and tropes.	Judging meaning ("would a stranger get this?") needs language understanding.
4 · LLM montage	Code cuts the strongest 5 "beats" from every episode (short windows with IDs like `E03–B2`). One request designs 4 trailers that each sell a different angle, referring to beats **by ID only**.	Story-level view across episodes, with no way to invent timestamps.
5 · Validate validate.py	Snaps each edge to the nearest real line (and flags it if it moved more than 2.5s), extends any edge that would cut through dialogue, merges overlaps, enforces 20–60s, checks for a real hook beat in the first 3s, and flags weak endings.	The LLM is never trusted with numbers. Code owns the timestamps.
6 · Score + dedupe rank.py	final = 0.75 × LLM score + 0.25 × rule score, minus penalties (context needed, warnings, length outside 25–45s), plus a small early-episode bonus. Promos sharing at least 50% of their footage are deduplicated.	One overconfident "10/10" can't carry a clip that has no hook.

7 · LLM judge	The top 20 go to one request that sets the **posting order** (variety, launch strategy) and calls out weak picks.	"What to post first" is a portfolio decision, not just a per-clip score.
8 · Editor	Reviews cards in the UI, approves or rejects with a reason, and exports EDL + ffmpeg + CSV.	A person stays accountable, and each click becomes data.

4 System design

4.1 Architecture

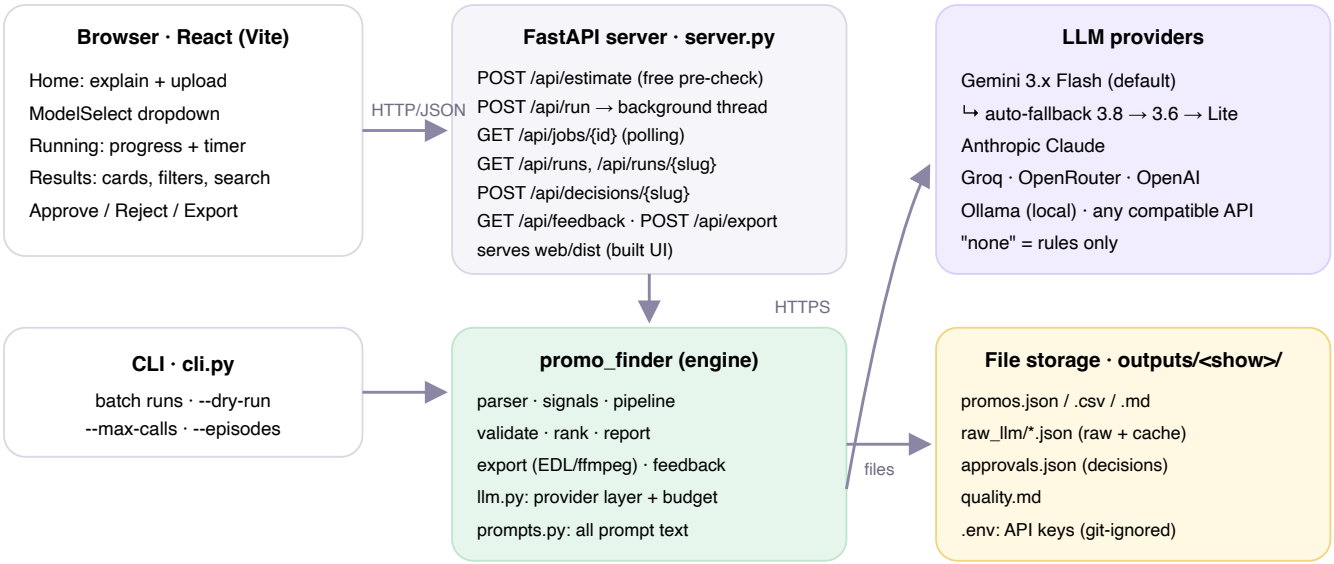


Figure 2. Components. The UI and the CLI are thin layers over the same engine, so anything the UI can do, a script can do too.

4.2 Component responsibilities

Module	Responsibility	Key detail
`parser.py`	Turns text into `Show → Episode → Scene/Event` dataclasses	Tolerant. Warnings instead of crashes
`signals.py`	Hook and cliffhanger scoring, candidate windows, top beats	Keyword lexicons + structure; non-max suppression on overlap
`llm.py`	One interface: `complete(system, user)` for 8 providers	Rate throttle, call budget, rate-limit retry, model fallback, SDK retries disabled
`prompts.py`	System, episode, montage and judge prompts	All in one file, easy to diff and tune
`pipeline.py`	Orchestration, the raw-response store and cache, progress callbacks	Cache key = hash(model + system + prompt)
`validate.py`	Promo data model; snapping and hard rules	Code owns timestamps
`rank.py`	Blended score, dedupe, judge ordering	Weights are explicit constants
`export.py`	EDL per promo, ffmpeg cut script, zip	Timecodes rounded half-up at a chosen fps
`feedback.py`	Saves decisions; computes precision and score gaps	Atomic file writes
`server.py`	REST API, background jobs, serves the UI	Reports which providers have keys (true/false only)

4.3 Data model

```
Show(title, episodes[])
 └─ Episode(number, logline, continuity, scenes[], events[], warnings[], duration)
     └─ Event(episode, scene, idx, kind: dialogue|action|camera|sfx|text, start, end, text,
speaker, emotion)

Promo(id, kind: single|multi|montage, title, segments[Segment(episode, start, end)],
      hook, cliffhanger, why, scores{hook, cliffhanger, standalone, emotion}, context_needed,
tropes[],
      source: llm|rules, rule_score, final_score, rank, rank_reason, flags[], excerpt[])

Decision (approvals.json): { promo_id: { status: approved|rejected, reason, at } }
```

4.4 Request lifecycle (web app)

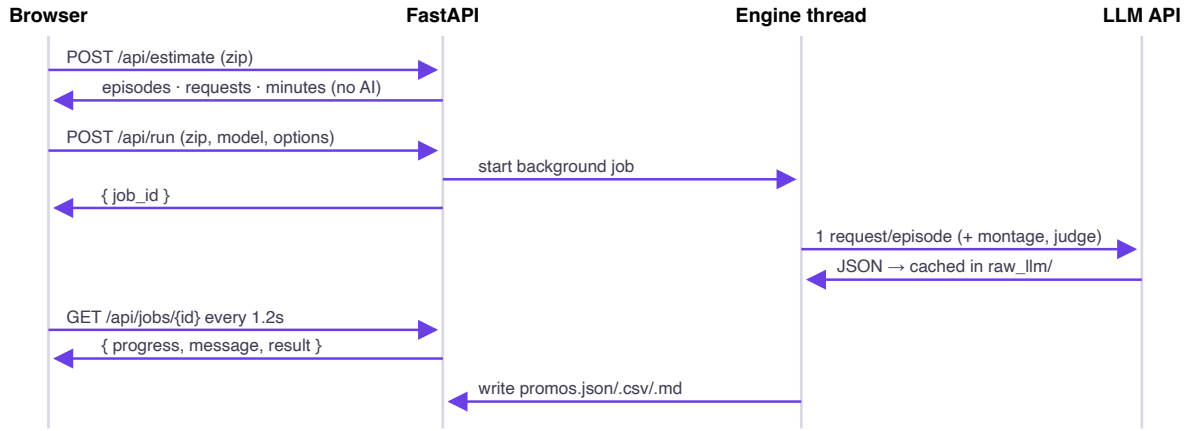


Figure 3. A run is a background job: the browser polls for progress, so a 20-minute run never hits an HTTP timeout.

4.5 Reliability and safety mechanisms

Graceful degradation

If the LLM fails for an episode, the rule candidates are used and the run log says so. A run always produces output.

Model fallback

If Gemini 3.8 Flash times out or is unavailable, the tool switches once to 3.6 Flash, then to Lite, and stays on the model that works.

Spend protection

`--dry-run` shows the cost first; `--max-calls` sets a hard budget; only rate limits are retried (at most twice); the SDK's hidden retries are switched off.

Cache

Every response is keyed by a hash of the model and prompt. Re-running the same show costs 0 requests, and the cache doubles as the raw-outputs folder.

No invented footage

Timestamps are snapped to real lines in code. Montage segments can only reference code-extracted beat IDs, and unknown IDs are flagged.

Secrets & confidentiality

Keys live in `.env` (git-ignored). The UI only learns which providers have a key, as true/false. `data/` is git-ignored. Uploads are processed in memory.

5 Key design decisions and trade-offs

Decision	Why	Trade-off accepted
Hybrid: rules + LLM + code checks	Rules are cheap and consistent but don't understand the story. The LLM understands the story but makes up numbers. Code checks keep both honest.	More moving parts than "one big prompt".
One LLM request per episode	Fits the context window easily, can be cached per episode, can run in parallel, and one failure costs only one episode.	Needs a separate montage pass for cross-episode promos.
Montages built from pre-cut beat IDs	The model chooses *which* moments; code decides *where* they are.	The model can't choose a moment the rules missed for montages.
75% LLM / 25% rules score blend	Keeps the ranking grounded, and the ranking still works with AI off.	The weights are judgement calls. Editor feedback is how we'd tune them.
Early-episode bonus (+0.03)	New viewers start at episode 1, so an ad's promise should pay off quickly.	May under-rank great late twists.
Files instead of a database	Easy to inspect, submit as raw outputs and version. No setup needed.	No multi-user concurrency or search. A database would come at production scale.
Background job + polling	Runs take minutes. Polling is simple and robust.	Job state is in memory and lost on restart (results already saved stay on disk).
Provider layer with "provider:model" strings	The brief rewards easy model swaps. Adding a provider means writing one method.	Lowest-common-denominator features (JSON mode only).
Gemini without Google Search tool	Promos must come only from the screenplay. Search adds latency and outside content.	None, for this task.
EDL + ffmpeg export	Fits the editor's existing tools (Premiere, Resolve), not a new one.	Assumes one video file per episode that matches the script.

6 Quality checking and known failures

How we know the output is good

- **Hard rules checked in code**: 20–60s, a strong hook beat in the first 3s, an ending on a cliffhanger cue, timestamps on real lines.
- **Coverage and variety**: episodes covered, promo types, tropes, and where the top 10 come from.
- **Rules vs LLM agreement**: picks the rules dislike are the first to eyeball.
- **Editor decisions**: precision overall and in the top 10, reject reasons, and which score separates good from bad.
- **Blind gold set**: pick your own promos before looking, then measure how many the tool found.

Where it picks badly (honest list)

- **Needs backstory**: late-episode villains and side characters confuse strangers.
- **Rules over-reward shouting**: the rules-only #1 pick was a thug's "Get out of the way!" (Ep 15).
- **Hook at the wrong end**: the rules placed the Ep 24 riding-crop slap at the *end* of a clip, when it should be the hook.
- **Quiet but devastating lines** are under-scored by keywords.
- **Montage continuity jumps** (costumes, locations) can't be seen in a script.

- **15 offline tests**, including an unseen script format and a fake LLM that returns deliberately sloppy output.
- **Blind to visuals**: a great slap with a weak line can lose to a strong line in a dull shot.

> **Caveat we state openly:** the automatic checks reuse our own rules, so they are partly circular. They catch broken picks but don't prove good ones. That's why the human review and the blind gold set matter.

7 Scale, cost and reliability

Metric	Value (one 50-episode show)	Notes
AI requests	**52** (50 episodes + 1 montage + 1 judge)	`--dry-run` prints this before spending
Input tokens	about **180K** + output	About 2.5–4K tokens per episode prompt
Time (one at a time, free tier)	about **9–39 min**	10–45s per request with medium thinking
Rerun cost	**0** requests	Cached by hash of the model and prompt
Rules-only mode	seconds, free	Instant baseline or fallback

Scaling to every show, every week

- **Parallelism:** episodes are independent, so on a paid tier they can run 10–20 at once. A show then takes minutes. A queue (Celery or Cloud Tasks) would replace the in-memory job thread.
- **Storage:** move from files to Postgres (runs, promos, decisions) plus object storage for raw responses once several editors work at the same time.
- **Cost control:** a cheap model (Flash Lite) for the per-episode pass and a stronger model only for the judge and montages. The rule shortlist can also skip episodes with weak signals.
- **Learning loop:** approve and reject decisions are used to re-weight scores. Later, real ad metrics (3-second hold rate, CTR, installs) become the target.
- **Next capability:** a vision model on the actual video to check the visual hook and the exact cut frames.

The reliability incident, and what it taught us

Symptom: a one-episode test sat at 0% for more than 4.5 minutes.

Root cause: `gemini-3.8-flash` was timing out on Google's side, *and* the Gemini SDK has hidden retries (4 extra attempts on timeouts and 5xx errors). Each attempt waited the full 150s timeout before our model fallback could run. An earlier version of our code also stacked its own 5 retries on top of a 3-model fallback, so one episode could have used about 15 requests.

Fix: SDK retries off (`attempts=1`), a 90s timeout, only rate limits retried, fall back once and stay on the working model, a call budget, a dry-run cost check, and live status messages in the UI ("3.8 is busy, switching…").

8 Tech stack and project layout

Layer	Technology
Language	Python 3.13
API server	FastAPI + Uvicorn

```
kuku-promo-finder/
├─ promo_finder/   engine (10 modules)
├─ cli.py          command line
├─ server.py       FastAPI app
├─ web/            React UI (src/, dist/)
├─ eval/           quality_check.py
```

UI	React 18 + Vite (vanilla CSS)
LLM SDKs	google-genai (Interactions API), openai, anthropic
Video	CMX3600 EDL, ffmpeg
Tests	unittest (standard library)
Config	.env via python-dotenv

```
├─ tests/          tests + odd_format_show
├─ docs/           solution.md, demo, guide
├─ outputs/        results per show
└─ data/           input pack (git-ignored)
```

9 How to run it

```
# setup (once)
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env              # add GEMINI_API_KEY
(cd web && npm install && npm run build)

# check before spending
python cli.py -i pack.zip --dry-run

# runs
python cli.py -i pack.zip --model none                          # rules only, free
python cli.py -i pack.zip --model gemini:gemini-3.6-flash --max-calls 60

# web app
python server.py                                                # http://localhost:8000

# quality, export, tests
python eval/quality_check.py outputs/<show>/promos.json -i pack.zip
python -m promo_finder.export outputs/<show>/promos.json --approved-only
python -m unittest discover -s tests -v
```

Swapping the model is one string: `--model`, `PROMO_MODEL` in .env, or the UI dropdown, e.g. `anthropic:claude-sonnet-5-5`, `groq:llama-3.3-70b-versatile`, `ollama:llama3.1`, or `compat:<model>` with `LLM_BASE_URL` for any OpenAI-compatible API Kuku provides.

10 Interview questions with model answers

The questions are grouped the way an interviewer usually moves: framing → design → AI → quality → scale → engineering → product → behavioural. Difficulty: Basic Intermediate Advanced. Answer in your own words. These are guides, not scripts.

A. Problem framing

Q1. Explain your project in 30 seconds. Basic

Editors at Kuku watch every episode to find promo clips, and new shows come out every week. I built a tool that reads the timestamped screenplays and returns a ranked list of 20–60s clips that open on a hook, end on a cliffhanger and make sense to a stranger. Each one has exact timestamps and a reason. Rules shortlist, an LLM picks and explains, code checks every timestamp, and an editor approves and exports to Premiere.

Q2. Why did you choose Problem 1 over the AI lawyer or your own problem? Basic

It's Kuku's core growth loop (paid acquisition), the input is structured (every line is timestamped) and the output can be checked (durations, timestamps, hook and cliffhanger). I could build something an editor could use the next day within 2–3 hours, and measure it honestly.

Q3. What assumptions did you make, and which one worries you most? Intermediate

That script time equals video time, that promos target strangers, that the hook must be a line or action, that we never cut mid-line, and that spoilers are fine. The one that worries me most is **script time = video time**. If the edit differs, every timestamp shifts. The fix would be aligning the script to the video's audio transcript before cutting.

Q4. How did you define a "good promo" in a way a program can check? Intermediate

I split it into checkable parts. Duration is 20–60s (code). A non-camera beat must start within the first 3s and score high on hook signals (code + LLM). The ending must have a cliffhanger cue or be the episode's final stinger (code + LLM). "Standalone" is judged by the LLM as a 1–10 score plus a "context needed" note. The parts code can't judge go to the LLM, and the final say goes to the editor.

B. System design

Q5. Walk me through the architecture. Basic

A Python engine (parse → rule signals → LLM per episode → montage → validate → score → judge) behind a FastAPI server and a CLI. The React UI talks to the server over JSON. A run is a background job the UI polls. Results, raw LLM responses and editor decisions are stored as files per show. LLM access goes through one provider layer, so the model is a single setting.

Q6. Why one LLM request per episode instead of sending all 50 episodes at once? Intermediate

Smaller prompts give more focused judgement, each episode can be cached separately, episodes can run in parallel, and one failure costs one episode, not the whole run. The trade-off is that cross-episode promos need a separate pass, which is why there's a montage step that sees every episode's loglines and best beats.

Q7. Why is a run a background job with polling? Why not just a normal request? Intermediate

A full show takes 10–40 minutes, which is longer than browser and proxy timeouts. A background thread with `/api/jobs/{id}` polling is simple and gives live progress. At scale I'd move to a proper queue (Celery or Redis, Cloud Tasks) so jobs survive restarts and can be spread across workers. Server-sent events or WebSockets would replace polling.

Q8. Why files and not a database? Basic

For a prototype, files are easy to inspect, version and hand in as "raw outputs", and need no setup. They break down with many editors writing at the same time and with search across shows. Then I'd move runs, promos and decisions to Postgres and raw responses to object storage. Decision writes are already atomic (write a temp file, then rename).

Q9. How does the system make sure the AI never gives a timestamp that doesn't exist? Advanced

The code owns timestamps. Every segment is snapped to the nearest real line start and end, and flagged if it moved more than 2.5s. If an edge falls inside dialogue, it's extended to the end of the line. Episodes that don't exist are rejected, and 20–60s is enforced. For montages the LLM can only cite beat IDs that code extracted, like `E03–B2`. Unknown IDs are flagged and dropped. Bad picks show up in a "Rejected" list instead of disappearing silently.

Q10. How would you change the design to handle 100 shows a week? Advanced

A queue with workers, episodes processed in parallel under a per-provider rate limiter, Postgres plus object storage, a cheap model for the per-episode pass and a strong model only for the judge, monitoring of cost and failure rate per show, and editor feedback feeding back into the score weights. The engine doesn't change. Only the orchestration and storage around it do.

C. AI / LLM engineering

Q11. What did you give to AI and what did you keep as code? Why? Basic

AI handles meaning: is this a hook, does it stand alone, why does it work, what order to post in. Code handles everything exact or cheap: parsing, timestamps, durations, deduplication and scoring arithmetic. Rules provide a shortlist and a fallback. LLMs are good at judgement and bad at precise numbers, so each part does what it's good at.

Q12. How do you get reliable JSON out of an LLM? Intermediate

JSON mode where the provider supports it (Gemini `response_format`, OpenAI `json_object`), an explicit schema in the prompt, and a tolerant parser that strips code fences and pulls the first JSON object out of surrounding text. Then every field is checked: bad timestamps are parsed or dropped, scores are clamped to 0–10. If parsing fails for an episode, it falls back to the rule picks for that episode.

Q13. How is the model "easy to swap"? Basic

The model is a `provider:model` string. `llm.py` has one small class per provider with a single `_call()` method. Gemini, Anthropic and OpenAI-compatible APIs (OpenAI, Groq, OpenRouter, Ollama, any custom endpoint) are supported, plus `none`. Prompts are provider-neutral and live in one file. If Kuku hands me an API in the interview, I set `compat:<model>` and a base URL.

Q14. How do you stop the model from hallucinating? Intermediate

I limit what it's allowed to make up. It only sees the script, with no web search. The system prompt says never invent lines or timestamps. Montages use only beat IDs. All numbers are re-checked in code. The rule score (25%) anchors the final score. The judge can flag weak picks. Every raw response is saved, so a hallucination can be traced and audited.

Q15. Why blend 75% LLM with 25% rules? Why not use the LLM's score alone? Advanced

LLM scores are often inflated and not well calibrated: many picks get 8–9/10. The rule score is a consistent second opinion based on real signals in the script, so a clip with no hook can't rank high just because the model was confident. The exact weights are a judgement call, and the editor feedback report shows which score actually separates approved from rejected picks, which is how I'd re-tune them with evidence.

Q16. What's in your prompts, and what made them better? Intermediate

The system prompt sets a persona (a performance-marketing editor for ReelShort- and DramaBox-style apps) and defines hook, cliffhanger and standalone with concrete examples. The episode prompt gives premise, logline, the full timestamped script and rule candidates ("may be wrong, use or ignore"), with exact rules (copy timestamps, 20–60s, hook in 3s) and a JSON schema. Asking for "context needed" made the model reveal its own standalone doubts, which feeds a penalty.

Q17. Why didn't you use the Google Search tool from Gemini's example code? Basic

Promos must come only from the show's own footage. Search adds latency and cost and could bring in outside content. It's the wrong tool for this job.

D. Quality and evaluation

Q18. How do you know your promos are actually good? Intermediate

Three layers. First, automatic hard-rule checks (duration, hook in 3s, cliffhanger ending, valid timestamps). Second, human review: approve or reject with reasons in the UI, giving precision overall and in the top 10. Third, a blind gold set: I picked promos myself before looking, then measured how many the tool found. I'm upfront that the automatic checks partly reuse my own rules. The real test is human judgement and, in production, ad metrics like 3-second hold rate.

Q19. Where does your tool pick badly? Basic

Clips that need backstory (late-episode villains), rules over-rewarding shouting (the rules-only #1 was a thug yelling "Get out of the way!"), a slap that should be the hook placed at the end of a clip, under-scored quiet lines, continuity jumps in montages, and anything visual, because it only reads the script.

Q20. What's precision vs recall here, and which matters more? Intermediate

Precision is the share of suggested promos that are actually good. Recall is the share of all good promos in the show that the tool found. For an editor's shortlist, **precision in the top 10** matters most, because they only post a few, and every bad pick wastes review time. Recall matters less, because a show has more good moments than you'll ever post. The blind gold set estimates recall. Approve and reject decisions measure precision.

Q21. How would you A/B test whether the tool's promos beat the editors' manual picks? Advanced

Pair promos from the same show, one tool pick and one editor pick, run them with the same budget, audience and placement, and compare 3-second hold rate, CTR, cost per install and day-7 retention of installers. Run enough impressions for statistical significance, and rotate which show and slot each gets to control for creative fatigue. Also measure editor time saved per show, since that's the original pain point.

E. Reliability, cost and scale

Q22. Tell me about a bug you hit and how you debugged it. Intermediate

A one-episode run sat at 0% for 4.5 minutes. The key and network were fine, because tiny test calls worked on 3.6 Flash. So I tested each model separately and saw 3.8 and 3.7 returning 504s. Then I read the SDK source and found hidden retries: 4 extra attempts on timeouts, each waiting 150s. I disabled SDK retries, cut the timeout to 90s, kept a single model fallback, and added live status messages, a call budget and a dry-run. The lesson: understand every retry layer, especially on a paid key.

Q23. What does it cost to run, and how did you protect a limited API key? Basic

52 requests and about 180K input tokens per 50-episode show. Protections: `--dry-run` shows the cost first, `--max-calls` sets a budget, a requests-per-minute throttle, retries only on rate limits, the cache makes reruns free, and the UI shows "About N AI requests · X–Y min" before you press run.

Q24. What happens if the AI provider is completely down? Basic

Gemini falls back across models. If every model fails, each episode falls back to rule candidates, and the montage and judge passes are skipped with a log entry. The editor still gets a ranked list, marked as rules-based, so it degrades rather than failing.

Q25. How would you make a full show finish in 2 minutes instead of 30? Advanced

Run episodes in parallel with a semaphore sized to the provider's rate limit (for example 20 at a time on a paid tier). Use lower thinking or a lighter model for the per-episode pass. Stream progress. Skip unchanged episodes through the cache. With 50 episodes at about 30s each and 20 in parallel, that's 3 rounds, about 1.5 minutes plus the montage and judge passes.

F. Software engineering

Q26. How did you test something that depends on a non-deterministic LLM? Intermediate

A fake LLM class returns canned, deliberately sloppy JSON: wrong timestamps, a too-short clip, unknown beat IDs, fenced output. The tests then check that the validation layer catches each problem, that the cache makes the second run cost 0 requests, and that the budget stops extra calls. All 15 tests run offline in about 4s.

Q27. What happens if the next show's scripts are formatted differently? Intermediate

The parser accepts MM:SS or HH:MM:SS, "." or "," decimals, bold or plain labels, and "Start/End", bracket or arrow timestamps. Broken blocks are skipped with warnings, and reversed times are swapped. I wrote an original test show in a different format to prove it. If a format is completely new, the parser is the only module to change, and the rest of the pipeline works on the same Event model.

Q28. Explain the EDL export. Why does it matter? Intermediate

An EDL (CMX3600) is a plain-text edit list that Premiere and Resolve import: each event has a source reel (the episode), source in/out and record in/out timecodes. I write one per promo, with the hook and cliffhanger as comments, and an ffmpeg script that cuts the real clips. It plugs into the editor's existing workflow instead of asking them to learn a new tool. I tested it on generated videos: 29.5s + 9.5s came out at exactly 39.0s. Testing also caught a rounding bug: Python's `round()` rounds halves to the nearest even number.

Q29. How do you handle API keys and confidential data? Basic

Keys live in `.env`, which is git-ignored. Keys typed into the UI are used for one run and never saved. The config endpoint only returns true/false for which providers have keys. The input pack sits in a git-ignored `data/`, uploads are processed in memory, and the test fixture is my own original content, so the repo can be public without leaking Kuku's show.

G. Product and human in the loop

Q30. Where does the human stay in the loop, and why? Basic

Nothing is posted automatically. The editor approves or rejects every pick, and can read the script excerpt and warnings first. Ads are brand-facing and cost money, and the tool can't see visuals. Each decision also becomes labelled data to improve the ranking.

Q31. How would you know the tool worked once a real team used it? Intermediate

Editor time per show (hours → minutes), approval rate of the top 10, the share of posted promos that came from the tool, and ad performance (3-second hold, CTR, CPI) against manually picked promos. Plus qualitative feedback: do the reject reasons shrink over time?

Q32. What would you build next with one more week? Intermediate

(1) A vision pass on the actual video to check visual hooks and exact cut frames. (2) Audio-transcript alignment so script time is checked against video time. (3) Learning the score weights from editor decisions, then from ad metrics. (4) Multi-language support (Kuku works across languages) by prompting in the show's language and evaluating with native editors. (5) Parallel processing and a queue.

Q33. Does this work for a Hindi or Tamil show? Advanced

The parser is language-neutral, because it keys on structure and timestamps. The keyword rules are English-only, so they'd weaken, and the score blend would lean more on the LLM. Modern LLMs read Hindi and Tamil well, but hooks are cultural, so I'd add language-specific lexicons, have the prompt explain local genre conventions, and validate with native-speaking editors before trusting the ranking.

H. Behavioural and AI-tool usage

Q34. Which AI tools did you use, and where did they get it wrong? Basic

I used Claude Code to scaffold the code, UI and docs. I made the scoping, assumption, weighting and review decisions. It got these wrong: it targeted `gemini-2.5-flash`, which returned 404 for new users because its knowledge was out of date. It sent `response_mime_type`, which the Interactions API rejects. And its first retry logic could burn about 15 requests per episode. I caught each one by running small tests and reading the errors. *(Add one example of your own from the real run.)*

Q35. How much of this is your own work? Could you change it live? Intermediate

Answer honestly: the AI wrote a lot of the code, and the decisions were mine. Show it by making a live change. For example, change `LLM_WEIGHT` in `rank.py`, add a reject reason, tighten the snap tolerance in `validate.py`, or add a keyword to `signals.py`. Then rerun and explain what moved and why. **Practise these four changes before the interview.**

Q36. If you had to cut this project to 1 hour, what would you keep? Intermediate

The parser, one LLM prompt per episode with JSON output, the code validation of timestamps and duration, and a Markdown report. That's the core value. The UI, montages, judge, export and feedback loop are all improvements on that core.

Tips for the interview

- Lead with the problem and the outcome, then the design. Use Figure 1 as your mental map.
- Bring numbers: 52 requests, about 180K tokens, 15 tests, your precision in the top 10, and how many of your gold-set picks the tool found.
- Volunteer the failures before you're asked. The brief rewards honesty over polish.
- Practise the "new show" demo (`docs/interview_demo.md`) and two or three live code changes.